

Development Plan

Invoice Processing — From PDF to Decision | One-Week Case Study Roadmap

1. Delivery Strategy

Build a modular monolith with a small, production-shaped architecture. The goal is not to build an enterprise AP platform in seven days; the goal is to demonstrate strong operational judgment with a workflow that actually runs, handles edge cases, and explains its decisions.

2. Priority Framework

Priority	Meaning	Examples
P0	Required for live demo	Upload, VLM extraction, PO matching, deterministic validation, duplicate checks, decision, run timeline
P1	Strong differentiator	Manual review, history filters, second verification, richer candidate evidence
P2	Post-case-study	Email ingestion, ERP connectors, queues, advanced analytics

3. Milestones

Day	Milestone	Deliverable
Day 1	Foundation	Repo, architecture, schema, fixtures, UI skeleton, provider abstraction
Day 2	Document understanding	PDF upload/rendering + VLM extraction + Pydantic schema
Day 3	PO intelligence	Vendor resolution + candidate retrieval + multi-signal matcher
Day 4	Controls	Arithmetic, tolerance, remaining balance, duplicate detection, decision engine
Day 5	UI integration	Live run timeline, decision detail, comparison UI, history
Day 6	Hardening	Edge cases, tests, error states, deployment, seeded demo data
Day 7	Submission	Final live test, 5-minute demo recording, clean README and backup fixtures

4. Backend Work Breakdown

- Create FastAPI project structure: api/, domain/, services/, providers/, models/, rules/, tests/.
- Implement file upload and secure temporary/object storage.
- Implement PyMuPDF rendering.
- Implement VLM adapter interface and provider implementation.
- Define strict invoice JSON schema.
- Implement normalization utilities for names, IDs, currency, dates and numeric values.

- Implement vendor resolver.
- Implement PO candidate retrieval.
- Implement weighted PO matcher.
- Implement deterministic validation rules.
- Implement duplicate service.
- Implement decision engine.
- Implement run event persistence and APIs.
- Implement review/override endpoint.

5. Database Build

1. Create PostgreSQL schema.
2. Seed 8–15 vendors and 15–30 POs covering clean, split, ambiguous and closed/cancelled cases.
3. Seed processed invoices for duplicate detection.
4. Create run/event tables.
5. Add indexes on vendor_id, PO number, invoice number, status, currency and normalized fields.
6. Add optional pgvector only if semantic retrieval is actually needed.

6. Frontend Build

7. Create dashboard shell and navigation.
8. Build upload screen.
9. Build live run timeline.
10. Build decision summary.
11. Build invoice/PO comparison table.
12. Build PO candidate cards.
13. Build review workspace.
14. Build history filters.
15. Implement loading, error, empty and review states.

7. AI Integration Plan

Use a provider interface so the application is not coupled to one API. The practical MVP can use a strong multimodal provider as the primary route and a compatible provider as a development/fallback route. Candidate providers discussed for this project include Gemini, OpenAI, OpenRouter and Groq; actual model availability, multimodal support, rate limits, free quotas and pricing must be checked at implementation time because provider offerings change.

- Call 1: invoice extraction from rendered page images.
- Call 2: optional verification only when required fields are uncertain or arithmetic/context is ambiguous.
- Do not ask the model to approve/reject.
- Do not send all POs to the model.
- Use model output for descriptions, references and ambiguous semantics; use code for money and policy.

8. Test Plan

Test Layer	Cases
Unit	Normalization, currency, rounding, tolerance, duplicate signatures, PO score
Schema	Valid/invalid model JSON, missing fields, unexpected types
Integration	Upload → extraction → matching → validation → decision
Regression	Fixed invoice/PO fixtures with expected decisions
UI	Upload, timeline, errors, review, comparison, history

Demo E2E	Happy path + at least 3 non-trivial edge cases
----------	--

9. Required Demo Dataset

Fixture	Purpose	Expected
INV-001 + PO-1001	Exact clean invoice	APPROVE
INV-002 + PO-1002	No PO number but unique vendor/amount	APPROVE
INV-003 + PO-1003/1004	Two plausible POs	REVIEW
INV-004 + PO-1005	Split invoice against remaining PO	APPROVE
INV-005 + PO-1006	Over tolerance	REVIEW/REJECT
INV-006 duplicate	Already processed	REJECT
INV-007 wrong vendor	PO reference points to another vendor	REVIEW/REJECT
INV-008 poor scan	Low-confidence critical field	REVIEW

10. Bug-Fixing Order

16. Fix decision-safety bugs first: anything that can incorrectly APPROVE.
17. Fix PO candidate correctness.
18. Fix arithmetic/tolerance errors.
19. Fix duplicate misses.
20. Fix extraction/schema failures.
21. Fix UI presentation issues.
22. Fix cosmetic issues last.

11. Deployment Checklist

- Frontend and backend deploy successfully from clean environment.
- Production environment variables configured.
- Model API key is server-side.
- Database migrations/seeding run successfully.
- Seeded PO/vendor data exists.
- Upload endpoint works over HTTPS.
- CORS restricted to frontend origin.
- Health endpoint returns healthy.
- A complete happy-path run works after deployment.
- At least three edge cases work after deployment.
- No secrets appear in repository or browser network payloads.
- README contains setup, architecture, demo fixtures and known limitations.

12. Definition of Done

23. A real PDF can be uploaded through the UI.
24. The system performs real multimodal extraction and returns structured fields.
25. The system can match an invoice to a PO without requiring PO-number equality.
26. The system handles zero, one and multiple PO candidates.
27. The system tracks remaining PO balance for split invoices.
28. The system independently validates arithmetic and tolerance.
29. The system checks duplicates.
30. The system produces APPROVE/REVIEW/REJECT through deterministic policy.

31. Every decision has human-readable reasons and evidence.
32. The live run view shows the actual stages as they execute.
33. At least four meaningful edge cases are tested.
34. The deployed application runs from a clean browser session.
35. The 5-minute demo can show happy path + edge case without manual database manipulation.

13. One-Week Scope Guardrails

- Do not build email ingestion before the core workflow works.
- Do not build payment execution.
- Do not build a full ERP connector.
- Do not build a custom OCR pipeline when the benchmark and case-study design support multimodal processing.
- Do not add microservices unless a real bottleneck appears.
- Do not add an agent that autonomously decides financial outcomes.
- Do not optimize for a flashy dashboard at the expense of matching correctness and auditability.

14. Final Demo Narrative

36. Start on dashboard and click Process Invoice.
37. Upload the clean invoice and narrate extraction.
38. Show vendor/PO resolution and the comparison.
39. Show deterministic checks passing and APPROVE.
40. Run an edge case with two plausible POs; show REVIEW and the candidate evidence.
41. Run a duplicate or tolerance violation; show that the system blocks automatic approval.
42. End by explaining the key design principle: AI handles messy document understanding and ambiguity; deterministic controls make the financial decision.